



01 • APERTURA

Ingeniería de Software I

UNIDAD VI • DIAGRAMAS DE CLASE

Prof. Lic. Guillermo Jacobo González Rodas Mst. PMP

Modelar la estructura de un sistema antes de implementarlo.



2 • OBJETIVOS

Objetivos

01

Comprender qué son los diagramas de clase y por qué son importantes.

02

Familiarizarse con las notaciones de los diagramas de clase.

03

Comprender la diferencia entre las relaciones de las clases.



3 • RECORRIDO

Contenido

01

Diagramas de Clase

02

Elementos

03

Clases y Objetos

04

Relaciones

05

Asociaciones

06

Herencia

07

Agregación y Composición

08

Dependencia y Modelo



4 • DEFINICIÓN

¿Qué es un Diagrama de Clase?

Los diagramas de clases son una herramienta para modelar sistemas.

Proporcionan una representación visual de las clases, atributos, métodos y las relaciones entre estas clases.

01

Documentan una solución de diseño: la estructura del sistema que se va a implementar en términos de clases y objetos.

02

Ayudan a documentar el modelado de datos.



5 • MODELO ESTÁTICO

Diagramas de Clase: ¿Qué Muestran?

Estructura estática

La estructura estática del sistema modelado, independiente del tiempo.

Relaciones

Las relaciones que existen entre las distintas clases y objetos del sistema.

Estructura interna

Las clases y objetos del sistema y su estructura interna.

No dinámicos

Muestran aspectos estáticos y no dinámicos; para el comportamiento temporal se usan otros diagramas.



6 • MODELO DE DOMINIO

Diagramas de Clase: ¿Para qué Sirven?

01

Realizar la abstracción de un dominio y formalizar el análisis de los conceptos relacionados con él.

Modelo de dominio

02

Definir y documentar una solución de diseño en términos de clases y objetos.

03

Definir y documentar el modelado de datos, como los diagramas ER.

Un modelo de dominio selecciona conceptos relevantes del problema; no es un inventario de todas las palabras del negocio.



07 • ELEMENTOS

Elementos de un Diagrama de Clases

Clase

Conjunto de objetos con características similares y comportamientos comunes.

Atributo

Propiedad que describe el estado; ejemplo: nombre: `String`.

Método

Comportamiento de una clase; ejemplo: `calcularPrecio(in precioBase: Double): Double`.

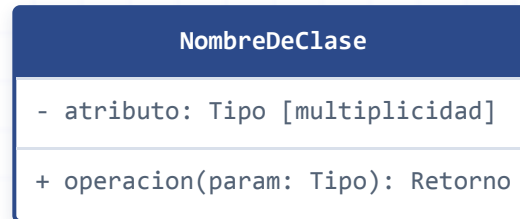
Relaciones

Conexiones entre clases y forma en que interactúan.



08 • NOTACIÓN DE CLASE

Las Clases



Una clase se representa mediante un rectángulo dividido en nombre, propiedades y operaciones.

Nombre: único identificador del clasificador.

Atributos: propiedades relevantes.

Operaciones: comportamiento disponible.

Sólo el nombre es obligatorio; los otros compartimentos pueden quedar vacíos.



9 · SINTAXIS

Atributos, Operaciones y Visibilidad

Visibilidad

+ Público · - Privado · # Protegido · ~ Paquete

Atributo

[visibilidad] nombre [:tipo] [multiplicidad] [=valor por omisión] [{propiedad}]

Operación

[visibilidad] nombre [(lista de parámetros)] [{propiedad}]

Parámetro

[dirección (in/out/inout)] nombre: tipo [multiplicidad] [=valor por omisión]



0 • INSTANCIAS

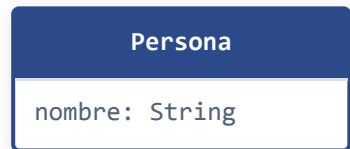
Clases y Objetos

Clase

Definición de estructura y comportamiento de un conjunto de objetos con el mismo patrón.

Objeto

Representación de algo mediante identificador, estructura y comportamiento. Es una instancia de una clase.



```
p1 = new Persona()
```

Instanciación: proceso de crear una instancia.





1 • TIPOS DE CLASE

Clases Parametrizables, Abstractas y Utilitarias

Parametrizables / Genéricas

Plantillas que se parametrizan con uno o más tipos de datos. Ejemplo: `Lista<T>`.

Abstractas

Clases que no tienen implementación para todos los métodos definidos y no se instancian directamente.

Utilitarias

Clases que contienen librerías de funciones; no interesa principalmente su implementación.





2 • RELACIONES

Las Relaciones y la Multiplicidad

Las relaciones representan conexiones entre clases y cómo interactúan entre sí.

La multiplicidad o cardinalidad representa el número de elementos de cada clase en una relación. Puede usar * para un número indeterminado.

0..1

cero o una instancia

1

exactamente una instancia

1..*

una o más instancias

0..*

cero o muchas instancias



3 • ASOCIACIONES

Asociaciones: 1:1, 1:N y N:M

1:1

Un Departamento tiene una Secretaria y una Secretaria pertenece a un Departamento.

1:N

Un Departamento tiene muchos Profesores; cada Profesor pertenece a un Departamento.

N:M

Un Estudiante cursa muchas Asignaturas y una Asignatura tiene muchos Estudiantes.

1

Departamento



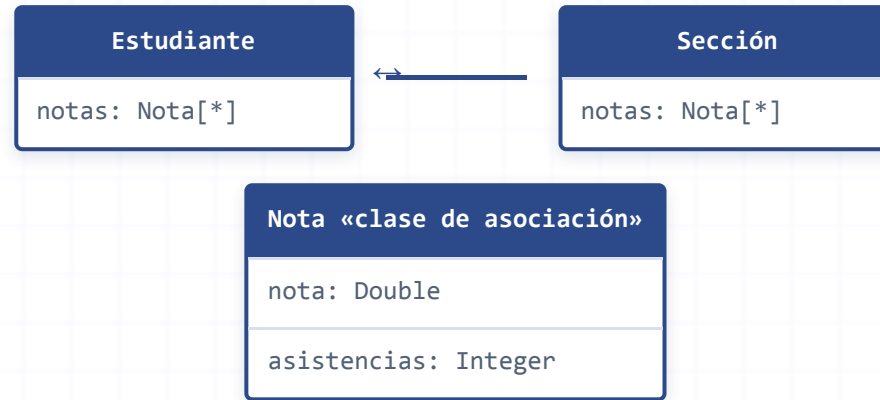
1..*

Profesor

La asociación se nombra con un verbo o rol: “tiene”, “pertenece”, “dicta”.

4 • ASOCIACIÓN CON DATOS

Clase de Asociación



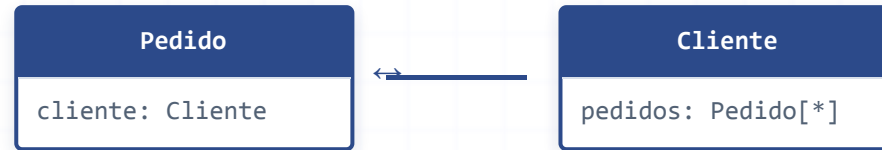
Una relación muchos a muchos se puede ver como dos relaciones uno a muchos.

La clase de asociación almacena datos del vínculo y mantiene referencias a las dos clases relacionadas.



5 • ASOCIACIONES DIRIGIDAS

Navegabilidad



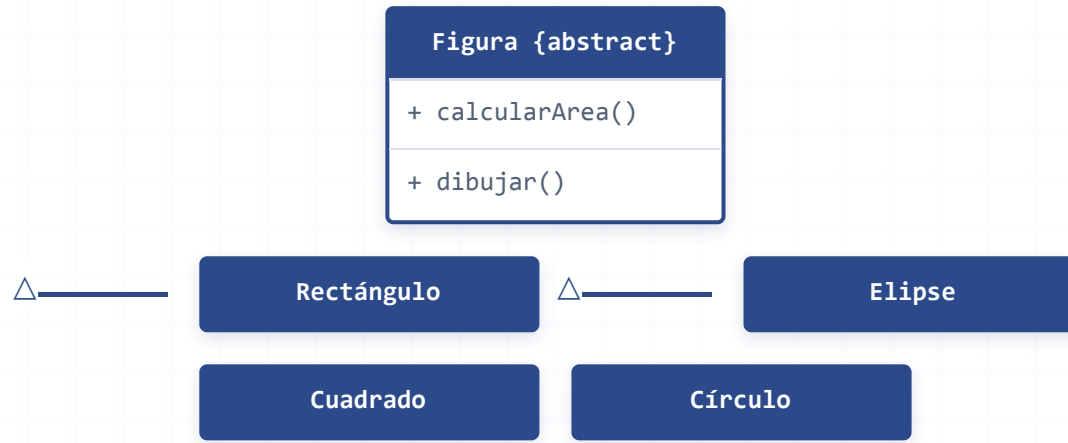
Navegable por ambos lados • Navegable • No navegable • Indefinido

La navegabilidad indica desde qué extremo se puede acceder o recorrer una asociación en el diseño.



6 • JERARQUÍA

Herencia, Generalización y Especialización



ES-UN(A): una subclase hereda atributos y operaciones de su superclase.

Restricciones: la herencia puede ser simple o múltiple; disjunta o traslapada; total o parcial.





7 • COMPORTAMIENTO

Polimorfismo y Reescritura

Polimorfismo

Se puede usar el mismo nombre para la definición de un método en varias clases.

Reescritura o sobrecarga

Permite nombrar código diferente con el mismo nombre para más de una clase de objetos o firma.

Enlace tardío

Selecciona el código adecuado al objeto definido en la invocación del método.

```
Figura figura = new Círculo();  
figura.dibujar();
```



8 • TODO-PARTE

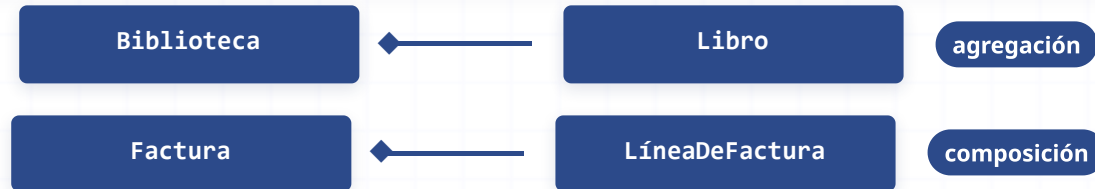
Agregación y Composición

Agregación

Una clase representa un todo y otra una parte. Se representa con un rombo blanco. La parte puede existir independientemente del todo.

Composición

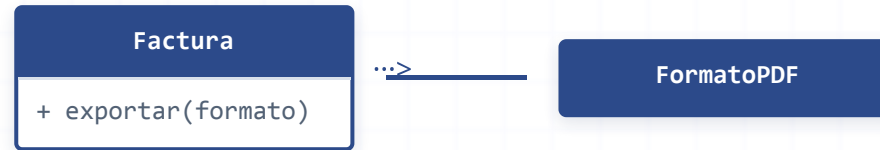
Es una forma más fuerte de todo-parte. Se representa con un rombo relleno. El todo tiene responsabilidad sobre la existencia de las partes.





9 • DEPENDENCIA Y CASO

Dependencia de Uso



La dependencia se representa con una línea punteada desde la clase que depende hacia la clase requerida.

Caso de Estudio: sistema de gestión para Emergencias del 911

Clases: **Zona**, **Tipo_Recurso**, **Emergencia**, **Tipo_Emergencia** y **Operador 911**.

Relaciones y multiplicidades: **Zona 1:N Emergencia**; **Tipo_Emergencia 1:N Emergencia**; **Operador 911 1:N Emergencia**; **Emergencia N:M Tipo_Recurso**.





20 • CIERRE

Criterios de Revisión

01

Cada clase se deriva de un concepto, requisito o responsabilidad.

02

Las asociaciones tienen nombres, multiplicidades y navegabilidad coherentes.

03

Las relaciones de herencia, agregación, composición y dependencia están justificadas.

04

El modelo es consistente con casos de uso y reglas; no agrega datos reales.

Un diagrama de clases vale por las decisiones que hace visibles.